

Session Hijacking

Demonstração local de sequestro de sessão por reutilização de cookie e mitigação com atributos seguros, expiração e invalidação no servidor.

Tema

05 - Session Hijacking

Integrantes

Eduardo
Otávio

Local

Dados fictícios

Express + express-session

Antes e depois

Objetivo e escopo ético

- Provar que uma sessão vulnerável pode ser reutilizada em outro cliente local.
- Mostrar que a correção reduz exposição e bloqueia cookies inválidos, antigos, expirados ou após logout.
- Comparar código vulnerável e código corrigido de forma didática.

Limite da demonstração

Somente 127.0.0.1/localhost, usuários fictícios (alice, bruno) e dados fictícios. Nada de contas reais, alvos externos, sniffing, XSS, CSRF ou túnel público.

Arquitetura do laboratório

Navegador

Login local e cookie de sessão enviado a cada request.



Express

/login, /dashboard, POST /logout e requireAuth.



express-session

Cookie guarda o ID; dados de sessão ficam no servidor.

Modo vulnerável

sid inseguro para provar o ataque.

Modo corrigido

__Host-session, HttpOnly, Secure, SameSite, maxAge.

Prova

npm test e demo manual com DevTools/cURL.

CONCEITO

Como acontece o Session Hijacking

1. Usuário legítimo autentica e recebe cookie de sessão.
2. Outro cliente obtém o mesmo identificador de sessão.
3. Servidor aceita o cookie e associa o segundo cliente à sessão existente.

Ideia central

Um cookie bearer, ou identificador de sessão ativo, funciona como credencial. Quem apresenta o cookie válido recebe a sessão enquanto ela existir.

O objetivo da mitigação é reduzir exposição, limitar a janela de uso e invalidar a sessão no servidor quando necessário.

Código vulnerável: cookie sid reutilizável

src/session/vulnerable-session.js

```
session({
  name: "sid",
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: false,
    secure: false,
    sameSite: false,
    maxAge: 24 * 60 * 60 * 1000
  }
});
```

Por que é vulnerável?

Vulnerabilidade demonstrada: reutilização de cookie de sessão sem nova autenticação.

- httpOnly: false aumenta exposição no cliente.
- secure: false permite envio sem HTTPS.
- sameSite: false remove restrição cross-site.
- maxAge longo amplia a janela de reuso.

Demonstração do ataque local

1. npm run dev.
2. Cliente A: login como alice.
3. Cliente B: /dashboard redireciona para /login.
4. Copiar sid do Cliente A e inserir no Cliente B.
5. Cliente B acessa /dashboard sem senha.

Fallback com cURL

```
curl.exe -i http://127.0.0.1:3000/dashboard ^  
-H "Cookie: sid=<copied-sid-value>"
```

Nos slides ou capturas de tela, redija o valor real do cookie.

Impacto e princípio violado

Impacto observado

- Segundo cliente acessa dados privados fictícios da Alice.
- Nenhuma senha é enviada no replay.
- O cookie passa a representar a identidade da sessão.

Princípio violado

Controle insuficiente sobre identificador de sessão: o servidor confia no cookie apresentado e não distingue o cliente legítimo do replay.

Evidência automatizada: `tests/session-reuse-attack.test.js` prova que uma requisição separada reutiliza sid e recebe dados fictícios da Alice.

Código corrigido: atributos seguros

src/session/fixed-session.js

```
session({
  name: "__Host-session",
  resave: false,
  saveUninitialized: false,
  cookie: {
    httpOnly: true,
    secure: true,
    sameSite: "strict",
    maxAge: 5 * 60 * 1000,
    path: "/"
  }
});
```

O que muda?

- HttpOnly: reduz exposição a JavaScript.
- Secure: envio apenas via HTTPS no caminho seguro.
- SameSite=Strict: restringe envio cross-site.
- maxAge curto: diminui janela de abuso.
- __Host-session: cookie mais restrito, com Path=/ e sem Domain.

Lifecycle: login, expiração e logout

Login

No modo corrigido, a sessão é regenerada antes de gravar `userId`.

Expiração

`maxAge` curto limita o tempo útil do cookie copiado.

Logout

POST `/logout` executa `req.session.destroy`, limpa o cookie e redireciona para `/login`.

O HTTP local com `npm run dev:fixed` é fallback de inspeção. Proteção real do atributo `Secure` exige HTTPS; os testes validam o caminho seguro com headers de proxy HTTPS.

Verificação: antes funciona, depois falha

Antes

Replay do sid vulnerável chega ao /dashboard e mostra Alice Demo / LAB-ALICE-001.

Depois

Cookie antigo, inválido, expirado ou após logout retorna 302 Location: /login.

```
npm test
```

```
mitigation verification:
```

- vulnerable sid replay reaches dashboard
- obsolete vulnerable sid is rejected in fixed mode
- copied fixed cookie fails after logout

OWASP e Secure SDLC

- OWASP Top 10 2025: A07 Authentication Failures.
- OWASP 2021: A07 Identification and Authentication Failures.
- OWASP Session Management: proteger o ID de sessão, expirar e invalidar.
- Secure SDLC: identificar, explorar de forma controlada, corrigir e verificar.

Conclusão e referências

Conclusão

O lab demonstrou, localmente, que um cookie de sessão inseguro pode ser reutilizado. Depois das correções, sessões inválidas ou encerradas não abrem mais o dashboard protegido.

Referências

- OWASP Top 10 A07 Authentication Failures.
- OWASP Session Management Cheat Sheet.
- MDN Set-Cookie.
- Express express-session.
- Código e testes deste laboratório.